

Cosmos+ OpenSSD Project#2

OpenSSD 펌웨어 코드 해킹 및
Block-Level FTL 구현 실습

임베디드시스템소프트웨어

담당 교수: 김영재

이름 / 학번: 한유승 / 20211607

수행 기간: 26.05.12 ~ 26.05.26

1. 프로젝트 개요 및 목표

- 본 프로젝트의 목표와 Task 흐름에 대해 서술하시오.

Greedy FTL의 데이터 저장 흐름을 이해하고, 기존의 page 단위 (slice 단위)의 매핑 테이블과 저장 방식을 block단위로 수정하여라.

2. 프로젝트 내용) Task1 – OpenSSD 내부 I/O 명령 처리 로직 분석

A. ReqTransNvmeToSlice 함수와 ReqTransSliceToLowLevel함수의 기능

- ... (OpenSSD에서의 `Slice` 용어 정의 설명 필수 포함)

Slice는 SSD의 기본 단위이며, SSD내에서의 page라고 보면 된다. Host에서 page는 4KB인 반면, slice는 16KB이다. 또한 LSA(Logical Slice Address)는 컴퓨터에서의 virtual address와 동일하고, VSA(virtual slice Address)는 컴퓨터에서의 physical address라고 생각하면 된다. 물론 실제로 VSA가 실제 NAND의 주소는 아니지만, VSA - PSA매핑은 하드웨어 단에서 일어나기 때문에, 사실상 VSA가 물리적 주소로 취급해도 된다.

ReqTransNvmeToSlice()

ssd내에서 기본 저장 단위가 slice이기 때문에, Request즉 명령도 Slice단위로 저장된다. 따라서 Host에서 request를 보내면, 그 request를 slice단위의 구조체로 변환하여 Nvme에게 전달해 줘야하는데, 이것이 ReqTransNvmeToSlice() 이다.

request slice는 총 세 과정을 거쳐 request queue에 큐잉된다.

- case 1: 먼저 첫 request slice를 생성하여 request queue에 할당해준다..
- loop: request가 slice보다 커질 수 있다. 따라서 몇 개의 slice가 생성될지 loop변수에 계산하여 기록해둔다.
- case2: loop가 0이 아니면 slice를 꽉 채울 수 있는 request가 있다는 뜻이다. 따라서 slice를 꽉 채운 request를 loop만큼 생성하여 queue slot에 각각 할당해준다.
- case3: 남은 request가 있지만, slice를 가득 채우지 못할 수 있다. 따라서 slice의 어디까지가 request라는것을 표시하여 request queue에 할당해준다.

ReqTransSliceToLowLevel()

ReqTransNvmeToSlice()에서 할당해준 request queue에서 request를 꺼내와서 해당 request를 NAND DMA로 내려주는 함수이다. 이 함수는 다음과 같은 처리 로직을 갖는다.

- request가 Dram hit시에는 바로 수행한다.
- request가 Dram Miss시에는 data buffer를 Dram에 할당해주고, data buffer에 NAND request를 작성한다.
 - Write 명령이면 reqCode = RxDMA, databuffer_dirty = true로 설정한다.
 - ◆ Rx: 호스트에서 쓰기 명령을 받았다는 뜻이다.
 - ◆ dirty: dram에서 evict될때, 변경사항이 있으니 NAND에 write하라는 뜻이다.
 - Read 명령이면 reqCode = TxDMA
 - ◆ Tx: 읽어서 호스트로 보내라는 뜻이다.

최종 정리

ReqTransNvmeToSlice(): Host Nvme요청을 FTL mapping 단위인 slice요청들로 분할해 request pool과 slice queue에 등록하는 함수이다.

ReqTransSliceToLowLevel(): request pool에 있던 request slice를 dram의 data buffer에 매핑한 뒤 필요하면 NAND read/write 요청을 추가로 만들고, 최종적으로 host-device DRAM간 전송을 위한 Nvme DMA요청인 RxDMA/TxDMA로 변환해 low-level request queue에 넣는 함수이다.

B. AllocateDataBuf 함수와 EvictDataBufEntry 함수의 기능

➤ ... (OpenSSD의 데이터 버퍼 구조 그림 필수 포함)

EvictDataBufEntry() 함수는 evict하고 재사용하려는 data buffer entry가 dirty 상태이면, NAND write request를 생성하여 low-level queue에 넣는 함수이다.

- dirty = false이면 NAND write-back없이 evict만 수행한다.
- dirty = true 이면 AddTransWrite()함수로 해당 LSA에 대한 새 VSA를

할당받고, REQ_TYPE_NAND, REQ_CODE_WRITE request를 만든다.

- 이 request는 data buffer entry의 내용을 NAND의 VSA위치에 써달라는 요청이 된다.
- write request를 queue에 넣은 뒤, 해당 data buffer entry의 dirty bit를 clean으로 바꾼다.
- 따라서 host write가 들어올 때 마다 NAND에 바로 쓰는 게 아니라, data buffer에 dirty만 표시하고, 해당 buffer entry가 재사용/evict될 때 lazy하게 NAND write-back이 발생한다.

AllocateDataBuf()

해당 함수는 data buffer에서 새 entry를 제공하기 위해 LRU방식으로 evict한 후 새 entry를 할당해주는 함수이다.

- data buffer LRU list의 tail entry를 victim으로 선택한다.
- victim을 LRU list의 tail에서 제거하고, head로 옮긴다.
- 이후 victim entry의 정보를 LSA – VSA 매핑 테이블에서 제거하여, 깨끗한 data buffer entry를 return한다.

C. AddrTransWrite 함수와 AddrTransRead 함수의 기능

AddrTransWrite()

해당 함수는 write request를 위해 LSA를 VSA와 매핑해주는 함수이다.

- 먼저 InvalidateOldVsa(LSA) 를 호출하여 기존의 LSA – VSA invalid mapping entry를 삭제한다.
- 그 다음 FindFreeVirtualSlice()로 새로운 free VSA를 하나 할당받는다.
- logicalSliceMap과 virtualSliceMap 각각에 LSA – VSA, VSA – LSA 매핑을 기록한다.
- 새로 할당된 새 VSA를 반환한다. 이렇게 out of place방식으로 기존 invalid physical page(virtual slice)를 업데이트 한다.

AddrTransRead()

해당 함수는 mapping table을 읽고 LSA를 VSA를 변환해주는 함수이다.

- logicalSliceMapPtr->logicalSlice[logicalSliceAddr].virtualSliceAddr을 가져온다.
- 해당 LSA에 VSA가 매핑 되어 있다면 그 VSA를 반환한다.
- valid한 VSA가 매핑 되어 있지 않다면 VSA_FAIL을 반환한다.

3. 프로젝트 내용) Task2 – GreedyFTL 코드 기반 Block-Level FTL 구현

A. Block-Level FTL 구현을 위한 address_translation.c 수정 내역

- ... (모든 수정 사항에 대해 before vs. after 형식으로 상세히 설명)
(수정 사항 설명 누락 발견 시 감점 가능)

1. address_translation.h 수정: LBN – VSA 매핑 테이블 추가

Before

기존 GreedyFTL은 page(slice) level FTL로, logicalSliceMap과 VirtualSliceMap만 존재한다.

즉, Logical Slice Address <-> Virtual Slice Address 1:1 매핑 구조였다.

After

Block level FTL을 위해 Block level mapping table을 먼저 선언했다.

```
// address_translation.h
typedef struct _LOGICAL_BLOCK_ENTRY {
    unsigned virtualBlockAddr;
} LOGICAL_BLOCK_ENTRY, *P_LOGICAL_BLOCK_ENTRY;

typedef struct _MY_LOGICLA_BLOCK_MAPPING_TABLE{
    LOGICAL_BLOCK_ENTRY logicalBlock[USER_BLOCKS_PER_SSD];
} MY_LOGICLA_BLOCK_MAPPING_TABLE,
*P_MY_LOGICLA_BLOCK_MAPPING_TABLE;

extern P_MY_LOGICLA_BLOCK_MAPPING_TABLE myBlockMapPtr;
```

2. address_translation.c 수정: 전역 변수 선언, 초기화.

Before

기존 GreedyFTL의 logical – virtual mapping table은 slice level만 존재했다.

After

block level logical – virtual mapping table을 추가하고, 해당 table을 초기화하는 InitMyLogicalMap()을 만들었다.

```
static MY_LOGICLA_BLOCK_MAPPING_TABLE myBlockMap;
P_MY_LOGICLA_BLOCK_MAPPING_TABLE myBlockMapPtr;

void InitMyLogicalMap()
{
    int blockAddr;
    for(blockAddr = 0; blockAddr < USER_BLOCKS_PER_SSD; blockAddr++){
        myBlockMapPtr->logicalBlock[blockAddr].virtualBlockAddr =
VSA_NONE;
    }
}

void InitAddressMap()
{
    unsigned int blockNo, dieNo;

    logicalSliceMapPtr = (P_LOGICAL_SLICE_MAP )
LOGICAL_SLICE_MAP_ADDR;
    ...
    myBlockMapPtr = &myBlockMap;
    ...
    InitSliceMap();
    InitBlockDieMap();

    InitMyLogicalMap();
}
```

myBlockMap을 static으로 선언한 이유

기존 GreedyFTL의 mapping 구조체들은 memory_map.h에 고정 DRAM주소로 선언되어 있고, myBlockMap 또한 이렇게 선언해야 FTL전반에서 사용 가능하다.

하지만 프로젝트의 수정 범위가 정확히 address_translation.c에 국한되어 있기 때문에, 해당 테이블을 GC와 같은 다른 범위(파일)에서 접근할 필요가 없다. 따라서 static으로 선언하여 file scope internal linkage를 부여하였다.

3. AddrTransRead()수정

Before

기존 코드는 LSA를 받아서 그대로 page level 매핑 테이블에 넣어 VSA를 반환한다. 즉 LSA – logicalSliceMap – VSA 구조였다.

After

수정된 코드에서는 LSA를 받아서 LBN + offset으로 분리하고, LBN – VBA 매핑 테이블 myBlockMap을 조회하여 VBA를 구하고, 이것과 offset을 이용하여 최종 VSA를 계산한다.

```
unsigned int AddrTransRead(unsigned int logicalSliceAddr)
{
    unsigned int virtualBlockAddr;
    unsigned int virtualSliceAddr;
    unsigned int dieNo;
    unsigned int VBN;
    unsigned int limit_offset;

    // Block => slice * 256 단위
    unsigned int LBN = logicalSliceAddr/256;
    unsigned int offset = logicalSliceAddr%256;

    if(LBN < USER_BLOCKS_PER_SSD){
        virtualBlockAddr = myBlockMapPtr->logicalBlock[LBN].virtualBlockAddr;
        if(virtualBlockAddr == VSA_NONE)
            return VSA_FAIL;
        else{
            // virtual == effectivly physical.
            dieNo = Vsa2VdieTranslation(virtualBlockAddr);
            VBN = Vsa2VblockTranslation(virtualBlockAddr);
            limit_offset = virtualBlockMapPtr->block[dieNo][VBN].currentPage;
            // offset 이 current page 넘어가면 그냥 current page 읽게. 안그러면
            // syscall 단에서 에러남.
            offset %= limit_offset;
            virtualSliceAddr =
Vorg2VsaTranslation(Vsa2VdieTranslation(virtualBlockAddr),
Vsa2VblockTranslation(virtualBlockAddr), offset);
        }
        if(virtualSliceMapPtr->virtualSlice[virtualSliceAddr].logicalSliceAddr == logicalSliceAddr)
            return virtualSliceAddr;
        else
            return VSA_FAIL;
    }
}
```

```

else
    assert(!"[WARNING] Logical address is larger than maximum logical
address served by SSD [WARNING]");
}

```

- LSA를 LBN과 offset으로 자른다. (SLICES_PER_SSD / USER_BLOCKS_PER_SSD = 블록당 slice개수 = 256)
- myBlockMapPtr->logicalBlock[LBN].virtualBlockAddr로 논리 블록을 VBA(virtual Block Address)로 변환한다.
- VBA에서 매크로를 이용해 die번호와 해당 die에서의 VBN(virtual block number)을 추출한다.
- vorg2VsaTranslation()에 die번호, VBN, slice offset을 넣어서 VSA를 만들어 반환한다.
- 이때, currentPag는 해당 블록에 실제로 기록된 페이지 수이다. offset이 이를 초과하면 syscall 단에서 에러가 난다. (실제로 확인해 봄.) 따라서 offset %= limit_offset으로 offset이 currentPage를 넘기지 못하게 한다.

3. AddrTransWrite()수정

Before

기존에는 FindFreeVirtualSlice()로 slice 단위로 free VSA를 매핑해주고 할당해 준다. 즉 LSA -> new VSA 구조이다.

After

Block level FTL은 block 전체를 하나의 단위로 관리하므로, GetFromFbList()로 free block자체를 할당하고 매핑해야 한다. 즉 LSA -> LBN -> new VBA 구조.

```

unsigned int AddrTransWrite(unsigned int logicalSliceAddr)
{
    unsigned int virtualBlockAddr;
    unsigned int virtualSliceAddr;
    unsigned int dieNo;
    unsigned int blockNo;
    unsigned int LBN = logicalSliceAddr / 256;
    unsigned int offset; // = logicalSliceAddr % 256 ... but need to be
sequential so...
}

```



```

if(LBN < USER_BLOCKS_PER_SSD)
{
    virtualBlockAddr = myBlockMapPtr->logicalBlock[LBN].virtualBlockAddr;

    if(virtualBlockAddr == VSA_NONE)
    {
        dieNo = sliceAllocationTargetDie;
        blockNo = GetFromFbList(dieNo, GET_FREE_BLOCK_NORMAL);

        if(blockNo == BLOCK_FAIL)
            assert(!"[WARNING] There is no available block [WARNING]");

        virtualBlockAddr = Vorg2VsaTranslation(dieNo, blockNo, 0);
        myBlockMapPtr->logicalBlock[LBN].virtualBlockAddr =
virtualBlockAddr;
        sliceAllocationTargetDie = FindDieForFreeSliceAllocation();
    }
    else
    {
        dieNo = Vsa2VdieTranslation(virtualBlockAddr);
        blockNo = Vsa2VblockTranslation(virtualBlockAddr);
    }

    if(virtualBlockMapPtr->block[dieNo][blockNo].currentPage ==
USER_PAGES_PER_BLOCK)
        assert(!"[WARNING] Current mapped block is full [WARNING]");

    // write sequentail !!!
    offset = virtualBlockMapPtr->block[dieNo][blockNo].currentPage;

    virtualSliceAddr = Vorg2VsaTranslation(dieNo, blockNo, offset);
    virtualSliceMapPtr->virtualSlice[virtualSliceAddr].logicalSliceAddr =
logicalSliceAddr;
    virtualBlockMapPtr->block[dieNo][blockNo].currentPage++;

    return virtualSliceAddr;
}
else
    assert(!"[WARNING] Logical address is larger than maximum logical
address served by SSD [WARNING]");
}

```

- LSA/256으로 LBN을 구한다.
- 먼저 기존의 LBN - VBA매핑 테이블에 매핑 전적이 있는지 확인한다.
- 매핑이 없는 경우

- ◆ sliceAllocationTargetDie로 할당 대상 die를 정하고, 해당 die의 free block list에서 빈 블록을 하나 가져온다.
- ◆ Vorg2VsaTranslation(dieNo, blockNo, 0)으로 VBA를 생성하고, LBN → VBA 매핑 테이블에 등록한다.
- ◆ 다음 할당을 위해 sliceAllocationTargetDie를 갱신한다.
- 매핑이 있는 경우
 - ◆ 기존 VBA에서 dieNo(VDN)과 blockNo(VBN)을 추출한다.
- 해당 블록의 currentPage가 USER_PAGES_PER_BLOCK에 도달했는지 확인한다. 블록이 가득 찼으면 assert로 중단한다.
 - ◆ 사실 제대로 block level FTL을 만들면, Block이 가득차면, 새 블록을 가져와서 LBN - VBA매핑을 갱신하고, 거기에 이어서 쓰고, GC를 중간중간 돌려가며 free block으로 victim block을 회수해야 한다. 하지만 과제 범위가 address translation으로 한정되어 있기 때문에, Assertion을 발생하도록 코딩하였다.
- offset을 currentPage로 설정한다. 즉 블록내에서 항상 순차적으로 다음 빈 페이지에 쓴다. 원래라면 block내에서 offset에 작성해야 하는데, 하드웨어적으로 block내에서 sequential write를 하지 않으면 죽어버리도록 설계되어 있어서, 어쩔 수 없이 currentPage에 작성해야한다.

B. ftl_bench 프로그램 기반 Block-Level FTL의 seq mode vs. rand mode 비교

- ... (seq 모드 테스트 결과 (호스트, 디바이스 모두) 캡처 필수 포함)

```

[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [19] [20] [21] [22] [23] [24] [25] [26] [27] [28] [29] [30] [31] [32] [33] [34] [35] [36] [37] [38] [39] [40] [41] [42] [43] [44] [45] [46] [47] [48] [49] [50] [51] [52] [53] [54] [55] [56] [57] [58] [59] [60] [61] [62] [63] [64] [65] [66] [67] [68] [69] [70] [71] [72] [73] [74] [75] [76] [77] [78] [79] [80] [81] [82] [83] [84] [85] [86] [87] [88] [89] [90] [91] [92] [93] [94] [95] [96] [97] [98] [99] [100] [101] [102] [103] [104] [105] [106] [107] [108] [109] [110] [111] [112] [113] [114] [115] [116] [117] [118] [119] [120] [121] [122] [123] [124] [125] [126] [127] [128] [129] [130] [131] [132] [133] [134] [135] [136] [137] [138] [139] [140] [141] [142] [143] [144] [145] [146] [147] [148] [149] [150] [151] [152] [153] [154] [155] [156] [157] [158] [159] [160] [161] [162] [163] [164] [165] [166] [167] [168] [169] [170] [171] [172] [173] [174] [175] [176] [177] [178] [179] [180] [181] [182] [183] [184] [185] [186] [187] [188] [189] [190] [191] [192] [193] [194] [195] [196] [197] [198] [199] [200] [201] [202] [203] [204] [205] [206] [207] [208] [209] [210] [211] [212] [213] [214] [215] [216] [217] [218] [219] [220] [221] [222] [223] [224] [225] [226] [227] [228] [229] [230] [231] [232] [233] [234] [235] [236] [237] [238] [239] [240] [241] [242] [243] [244] [245] [246] [247] [248] [249] [250] [251] [252] [253] [254] [255] [256] [257] [258] [259] [260] [261] [262] [263] [264] [265] [266] [267] [268] [269] [270] [271] [272] [273] [274] [275] [276] [277] [278] [279] [280] [281] [282] [283] [284] [285] [286] [287] [288] [289] [290] [291] [292] [293] [294] [295] [296] [297] [298] [299] [300] [301] [302] [303] [304] [305] [306] [307] [308] [309] [310] [311] [312] [313] [314] [315] [316] [317] [318] [319] [320] [321] [322] [323] [324] [325] [326] [327] [328] [329] [330] [331] [332] [333] [334] [335] [336] [337] [338] [339] [340] [341] [342] [343] [344] [345] [346] [347] [348] [349] [350] [351] [352] [353] [354] [355] [356] [357] [358] [359] [360] [361] [362] [363] [364] [365] [366] [367] [368] [369] [370] [371] [372] [373] [374] [375] [376] [377] [378] [379] [380] [381] [382] [383] [384] [385] [386] [387] [388] [389] [390] [391] [392] [393] [394] [395] [396] [397] [398] [399] [400] [401] [402] [403] [404] [405] [406] [407] [408] [409] [410] [411] [412] [413] [414] [415] [416] [417] [418] [419] [420] [421] [422] [423] [424] [425] [426] [427] [428] [429] [430] [431] [432] [433] [434] [435] [436] [437] [438] [439] [440] [441] [442] [443] [444] [445] [446] [447] [448] [449] [450] [451] [452] [453] [454] [455] [456] [457] [458] [459] [460] [461] [462] [463] [464] [465] [466] [467] [468] [469] [470] [471] [472] [473] [474] [475] [476] [477] [478] [479] [480] [481] [482] [483] [484] [485] [486] [487] [488] [489] [490] [491] [492] [493] [494] [495] [496] [497] [498] [499] [500] [501] [502] [503] [504] [505] [506] [507] [508] [509] [510] [511] [512] [513] [514] [515] [516] [517] [518] [519] [520] [521] [522] [523] [524] [525] [526] [527] [528] [529] [530] [531] [532] [533] [534] [535] [536] [537] [538] [539] [540] [541] [542] [543] [544] [545] [546] [547] [548] [549] [550] [551] [552] [553] [554] [555] [556] [557] [558] [559] [560] [561] [562] [563] [564] [565] [566] [567] [568] [569] [570] [571] [572] [573] [574] [575] [576] [577] [578] [579] [580] [581] [582] [583] [584] [585] [586] [587] [588] [589] [590] [591] [592] [593] [594] [595] [596] [597] [598] [599] [600] [601] [602] [603] [604] [605] [606] [607] [608] [609] [610] [611] [612] [613] [614] [615] [616] [617] [618] [619] [620] [621] [622] [623] [624] [625] [626] [627] [628] [629] [630] [631] [632] [633] [634] [635] [636] [637] [638] [639] [640] [641] [642] [643] [644] [645] [646] [647] [648] [649] [650] [651] [652] [653] [654] [655] [656] [657] [658] [659] [660] [661] [662] [663] [664] [665] [666] [667] [668] [669] [670] [671] [672] [673] [674] [675] [676] [677] [678] [679] [680] [681] [682] [683] [684] [685] [686] [687] [688] [689] [690] [691] [692] [693] [694] [695] [696] [697] [698] [699] [700] [701] [702] [703] [704] [705] [706] [707] [708] [709] [710] [711] [712] [713] [714] [715] [716] [717] [718] [719] [720] [721] [722] [723] [724] [725] [726] [727] [728] [729] [730] [731] [732] [733] [734] [735] [736] [737] [738] [739] [740] [741] [742] [743] [744] [745] [746] [747] [748] [749] [750] [751] [752] [753] [754] [755] [756] [757] [758] [759] [760] [761] [762] [763] [764] [765] [766] [767] [768] [769] [770] [771] [772] [773] [774] [775] [776] [777] [778] [779] [780] [781] [782] [783] [784] [785] [786] [787] [788] [789] [790] [791] [792] [793] [794] [795] [796] [797] [798] [799] [800] [801] [802] [803] [804] [805] [806] [807] [808] [809] [810] [811] [812] [813] [814] [815] [816] [817] [818] [819] [820] [821] [822] [823] [824] [825] [826] [827] [828] [829] [830] [831] [832] [833] [834] [835] [836] [837] [838] [839] [840] [
```

```
result: OK = 10000 FAIL = 0
```

- ... (rand 모드 테스트 결과 (호스트, 디바이스 모두) 캡처 필수 포함)

```
5월 19일 21:47
github
embe2024@embe2024-PNVKB0A0-Samsung-DeskTop: ~/wor...
embe2024@embe2024-PNVKB0A0-Samsung-DeskTop:~$ cd workspace/
embe2024@embe2024-PNVKB0A0-Samsung-DeskTop:~/workspace$ ./setup.sh GreedyFTL sca
Scheme: GreedyFTL
TYPE: scan
Xilinx FPGA board is detected at 0000:01:00.01
[sudo] embe2024 암호:
embe2024@embe2024-PNVKB0A0-Samsung-DeskTop:~/workspace$

embe2024@embe2024-PNVKB0A0-Samsung-DeskTop: ~/workspace/prj2-materials/prj2...
embe2024@embe2024-PNVKB0A0-Samsung-DeskTop:~/workspace/prj2-materials/
embe2024@embe2024-PNVKB0A0-Samsung-DeskTop:~/workspace/prj2-materials$ cd prj2-host-evaluation/
embe2024@embe2024-PNVKB0A0-Samsung-DeskTop:~/workspace/prj2-materials/prj2-host-evaluation$ sudo .fft
l_bench /dev/memdev1 rand 1000 1048576 1
[sudo] embe2024 암호:
-----
Cosmos+ OpenSSD FTL Benchmark (rand)
mode:rand ops=1000 span=1048576 (10KB unts)
result: DWG42 FAIL=350
elapsed: 18996.7 ms (182.404 IOPS est. for R/W)
embe2024@embe2024-PNVKB0A0-Samsung-DeskTop:~/workspace/prj2-materials/prj2-host-evaluation$

embe2024@embe2024-PNVKB0A0-Samsung-DeskTop: ~/wor...
Done Admin Command OPC: 6
-----
[FTL] Mapping Table Summary
-----
Valid logicalSliceMap entries : 0 / 4194304
Valid virtualSliceMap entries : 0 / 4194304
Unique mapped blocks : 0 / 16384
-----
[FTL] Space Utilization
-----
Free blocks remaining : 16376
-----
[FTL] Mapping Table Summary
-----
Valid logicalSliceMap entries : 0 / 4194304
Valid virtualSliceMap entries : 17384 / 4194304
Unique mapped blocks : 0 / 16384
-----
[FTL] Space Utilization
-----
Free blocks remaining : 15678
-----
```

result: OK = 642, FAIL = 358

- ... (평가 결과에 대한 해석 필수 기재)

일단 Test bench에 같은 Logical Block을 가득 채워서 새 block을 할당시키게 하는 케이스는 없기 때문에, GC를 건드리지 않고 과제를 해결할 수 있다.

Sequential

- ◆ sequential모드에서는 LBA가 0, 4, 8, 12, ... 으로 순차 증가한다. Block level FTL에서 하나의 LBN내 offset이 순서대로 채워지므로, currentPage와 write 순서가 일치하여 모든 검증을 통과한다.
- ◆ read에서도 offset이 currentPage와 일치하게 증가하기 때문에, 모든 검증을 통과한다.

Random

- ◆ random write에서는 같은 LBN내에 들어오는 write의 순서(offset)이 뒤죽박죽이다. 예를들어 LBN 1에 offset 200 - 3 - 150순서로 write가 와도, openSSD cosmos+ NAND FPGA자체의 sequential write 제약¹ 때문에 실제로는 currentPage 0 - 1 - 2 순서로 기록해야 하며, 실제로 currentPage++에 기록하고 있다.
- ◆ 이때, read시에는 $\text{offset} = \text{LSA} \% 256$ 으로 계산하여 블록의 200 - 3 - 150을 읽으려 하는데, 실제 데이터는 물리 페이지 (VSA) 0,1,2에 있다. 따라서 논리적 offset과 물리적 기록 위치가 불일치하게 되고, 잘못된 데이터를 읽게 된다.
더 심각한 점은, 만약 offset이 currentPage범위를 넘겨버리면, 그대로 하드웨어 단에서 crash가 발생해 중지되게 된다. 이를 방지하기 위해 $\text{offset} \% = \text{currentPage}$ 로 범위초과는 막았지만, 이는 정말 하드웨어 crash만 막는 역할을 한다.

C. 현재 Block-Level FTL 구현체의 제한사항 (문제점)과 보완 계획 서술

- ... (예시: 문제 - rand mode에서 데이터 정합성이 맞지 않음
보완 계획 - blahblah ...)

¹ S. Jung, S. Lee, J. Han, and Y. Kim, "Preemptive Zone Reset Design within Zoned Namespace SSD Firmware," Electronics, vol. 12, no. 4, pp. 798, 2023.

데이터 정합성 문제

위에서 말했듯이 rand mode에서는 input offset과 실제 물리적으로 기록된 offset이 달라 발생하는 데이터 정합성 문제가 발생한다. 이를 완전히 해결하기 위해서는 하드웨어적으로 block 내에서 random write를 허가해야 하는데, 이는 openSSD cosmos+보드의 sequential write 제약에 위배되어 불가능하다.

따라서 log block 기법을 도입해야 한다. random write 요청이 들어오면, 먼저 별도의 log block에 currentPage++ 순서로 순차 기록하되, 각 엔트리의 논리적 offset과 물리적 page 위치를 매핑하는 테이블을 함께 관리한다. 이후 log block이 가득 차면, 이 매핑 테이블을 참조하여 원래 data block과 merge하면서 올바른 offset 순서대로 새 블록에 순차 기록한다. 이를 통해 read 시 정확한 위치에서 데이터를 찾을 수 있게 된다.

블록이 꼭 찾을 때의 문제

현재 구현에서는 GC나 새로운 블록 할당을 지원하지 않는다. 따라서 같은 LBN에 대한 write가 반복되어 해당 블록의 currentPage가 USER_PAGES_PER_BLOCK에 도달한 후, 한 번 더 write 요청이 오면 assert로 펌웨어가 중단된다. 이를 해결하려면 free block list에서 새 블록을 할당하여 LBN 매핑을 갱신하고, 기존 블록의 valid 데이터를 새 블록으로 복사한 뒤 기존 블록을 erase하여 회수하는 GC 로직을 구현해야 한다.

실제로 10000개의 sequential test 이후 random test를 하려고 하면, 해당 문제가 발생한다. 아래는 사진이다.

```
[*] 테이퍼 > embe2024@embe2024-PHVXB0A0-Samsung-DeskTop :~$ workspace
embe2024@embe2024-PHVXB0A0-Samsung-DeskTop :~$ workspace Q - - - - - X
embe2024@embe2024-PHVXB0A0-Samsung-DeskTop :~$ cd workspace/
embe2024@embe2024-PHVXB0A0-Samsung-DeskTop :~/workspace$ ./setup.sh GreedyFTL scan
Scheme: GreedyFTL
Type: scan
Xilinx spca board is detected at 8000:01:00:01!
[sudo] embe2024 암호:
embe2024@embe2024-PHVXB0A0-Samsung-DeskTop :~/workspace$ ls
GreedyFTL README.md prj1-materials prj2-materials setup.sh util
embe2024@embe2024-PHVXB0A0-Samsung-DeskTop :~/workspace$ cd prj2-materials/
embe2024@embe2024-PHVXB0A0-Samsung-DeskTop :~/prj2-materials$ ls
prj2-document.docx prj2-host-evaluation prj2-workspace-patch task1.md task2.md test.py
embe2024@embe2024-PHVXB0A0-Samsung-DeskTop :~/prj2-materials$ cd prj2-host-evaluation/
embe2024@embe2024-PHVXB0A0-Samsung-DeskTop :~/prj2-materials/prj2-host-evaluation$ ls
Makefile README.md rtl_bench ftl_bench.cc mve_passthru.cc mve_passthru.o
embe2024@embe2024-PHVXB0A0-Samsung-DeskTop :~/prj2-materials/prj2-host-evaluation$ make
make: "rtl_bench" 옵션이 이미 업데이트되었습니다.
embe2024@embe2024-PHVXB0A0-Samsung-DeskTop :~/prj2-materials/prj2-host-evaluation$ sudo .ftl
l_bench $dev/nvme0n1 seq 10000 1048576 1
-----
Free blocks remaining      : 16376
-----
[FTL] Mapping Table Summary
-----
Valid logicalSlicemap entries : 0 / 4194904
Valid virtualSlicemap entries : 26384 / 4194904
Unique mapped blocks       : 0 / 16384
[FTL] Space Utilization
-----
Free blocks remaining      : 16372
-----
[FTL] Mapping Table Summary
-----
Valid logicalSlicemap entries : 0 / 4194904
Valid virtualSlicemap entries : 26384 / 4194904
Unique mapped blocks       : 0 / 16384
[FTL] Space Utilization
-----
Free blocks remaining      : 16372
-----
assertion "[WARNING] Current mapped block is full [WARNING]" failed: file "../src/address_translation.c", Line 741, Function: AddrTranslate
WONG Cc-Bit 0
PCIE MSI Enable: 0, 0x3
PCIE IOQ disable: 1
PCIE AUR Master: 0
PCIE BUS Master: 1
PCIE IRQ disable: 0
PCIE MSI Enable: 1, 0x0
PCIE MSI Enable: 0, 0x0
PCIE IRQ disable: 1
PCIE IOQ disable: 1
PCIE BUS Master: 0
```

D. 보완된 Block-Level FTL을 Page-Level FTL과 비교했을 때, seq mode와 rand mode에서의 예상 성능 서술

- ... (예상되는 성능 평가 결과에 대한 이유를 개념과 연관지어 설명하시오)
- ... (예시: 메모리 사용량, GC 빈도, throughput 등)

Sequential mode

seq mode에서는 두 FTL 간 성능 차이가 거의 없을 것으로 예상된다. sequential write 시 log block에 기록되는 순서와 논리적 offset이 자연스럽게 일치하므로, log block merge 과정에서 추가적인 재배치가 불필요하다. Page-level FTL도 순차 접근 시 매핑 테이블의 공간 지역성이 높아 캐시 효율이 좋기 때문에, 두 방식 모두 유사한 throughput을 보일 것이다. 다만 메모리 사용량에서는 block-level FTL이 유리하다. page-level FTL은 모든 페이지에 대해 개별 매핑 엔트리를 유지해야 하지만, block-level FTL은 블록 단위 매핑과 log block 내 소규모 매핑 테이블만 관리하면 되므로 DRAM 사용량이 훨씬 적다.

Random mode

rand mode에서는 page-level FTL이 throughput 면에서 우세할 것으로 예상된다. page-level FTL은 각 페이지의 물리 위치를 독립적으로 추적하므로, random write가 들어와도 빈 페이지에 바로 기록하고 매핑만 갱신하면 된다. 반면 보완된 block-level FTL은 log block이 가득 찰 때마다 data block과의 merge가 필요하고, 이 merge 과정에서 valid 페이지를 읽어서 새 블록에 copy & write 추가 I/O가 발생한다. 이로 인해 write amplification이 증가하고 throughput이 저하된다.

GC 빈도 측면에서도 block-level FTL이 불리하다. log block merge 자체가 새 블록을 소모하고 기존 블록을 erase해야 하므로, 같은 양의 write에 대해 더 많은 erase가 발생한다. page-level FTL은 invalid page가 충분히 쌓인 victim block을 골라 GC하므로, erase 빈도를 상대적으로 줄일 수 있다.

그러나 메모리 사용량에서는 여전히 block level FTL이 유리하다. Cosmos+ OpenSSD 기준으로 page-level 매핑 테이블은 약 128MB의 DRAM을 차지하지만, block-level FTL은 블록 매핑 테이블과 log block 매핑만 유지하면 되므로 수 MB 이내로 관리할 수 있다. 이는 DRAM 용량이 제한된 임베디드 SSD 환경에서 의미 있는 이점이다.

E. Hybrid FTL의 필요성 서술

➤ ... (Page-Level FTL, Block-Level FTL과 연관지어 설명하시오)

Page-level FTL은 모든 페이지에 대해 개별 매핑 엔트리를 유지하므로 random write 성능이 우수하지만, 매핑 테이블이 지나치게 크다. 1TB SSD 기준으로 4KB 페이지마다 4B 엔트리를 두면 매핑 테이블만 1GB에 달하며, 이는 SSD 내장 DRAM의 대부분을 차지하게 된다. 반면 block-level FTL은 블록 단위로만 매핑하여 테이블 크기를 수십 MB 수준으로 줄일 수 있지만, 한 페이지만 갱신하는 small random write request 에도 같은 블록의 나머지 valid 페이지를 모두 읽어 새 블록에 복사해야 하므로 write amplification이 심하다.

Hybrid FTL은 이 두 방식의 장점을 결합한다. 기본적으로 data block을 block-level 매핑으로 관리하여 DRAM 사용량을 낮게 유지하고, overwrite(update)가 발생할 때만 별도의 log block에 page-level 매핑으로 기록한다. Read 시에는 log block의 page 매핑 테이블을 먼저 확인하고, 해당 엔트리가 없으면 data block의 block 매핑 테이블을 참조한다. Log block이 가득 차면 data block과 merge를 수행하는데, 상황에 따라 full merge, partial merge, switch merge 중 적절한 방식이 선택된다. 특히 sequential update의 경우 switch merge로 erase 한 번만에 처리할 수 있어 효율적이다.

이를 통해 sequential workload에서는 data block에 직접 기록하여 log block 소모 없이 block-level FTL과 동등한 효율을 얻고, random workload에서는 log block이 page-level FTL의 역할을 대신하여 병목을 회피할 수 있다. 매핑 테이블 크기도 block-level 매핑 테이블에 소수의 log block에 대한 page-level 매핑만 추가되므로, page-level FTL 대비 크게 절감되어 DRAM이 제한된 SSD에서 사용하기에 좋다.

4. 참고 문헌

- 보고서 작성 간 최소 하나 이상의 공인 학술 문헌 (e.g., 논문, 기술 리포트, 기술 Specification 등)을 참조하시오.

S. Jung, S. Lee, J. Han, and Y. Kim, "Preemptive Zone Reset Design within Zoned Namespace SSD Firmware," Electronics, vol. 12, no. 4, pp. 798, 2023. (논문)

Y. Kim, "NAND Flash Management (2) - Address Translation," Embedded System Software (CSE4116-01), Sogang University. (강의자료)